

Graphs in Machine Learning

— Spring Quarter, Lecture 2 —

Prof. Jeff Bilmes

University of Washington, Seattle
Departments of: Electrical & Computer Engineering, and Computer Science & Engineering
<http://melodi.ee.washington.edu/~bilmes>

April 2nd, 2025

Class Road Map/Progress - EE512 - Spring 2025

- L1 (3/31): Introduction, Overview, Graphs, Graphical Models, Graph Neural Networks
- L2 (4/2): Graphs Theory, Trees, Node and Edge Statistics, Hyper-Trees
- L3 (4/7):
- L4 (4/9):
- L5 (4/14):
- L6 (4/16):
- L7 (4/21):
- L8 (4/23):
- L9 (4/28):
- L10 (4/30):
- L11 (5/5):
- L12 (5/7):
- L13 (5/12):
- L14 (5/14):
- L15 (5/19):
- L16 (5/21):
- LXX (5/26): Memorial day holiday
- L17 (5/28):
- L18 (6/2):
- L19 (6/4):
- Final Presentations due: (Friday 6/13/2023):

Holidays: Monday 6/26 (Memorial day). Last day of instruction: Fri, 6/7. Final Exam Week: 6/7-6/13. Our Exam Time: Fri, June 13th, 2:30 - 4:30 PM, ECE 267.

Goals

- Our goal is: 25% graphs, 37.5% graphical models, 37.5% graph neural networks.

- Our goal is: 25% graphs, 37.5% graphical models, 37.5% graph neural networks.
- Our goal for the next roughly 4.5 lectures is to study some relative graph theory useful both for graphical models or graph neural networks, but also study some ways that graphs have been used historically in machine learning (e.g., without much, or any, use of neural methods).

Graphs

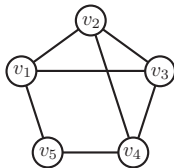
- A graph is a set of nodes V and a set of edges E that indicate a relation between (typically) two nodes. “edges” and “links” are synonymous in this class.

Why Graphs (and why Graphs in ML)?

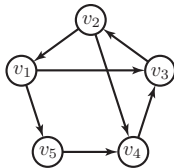
- Graphs are a general, visual, (often) mathematically precise, language for describing and modeling data, complex systems, processing thereon, and relationships therein.

Matrix representation of graphs

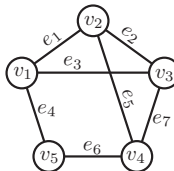
- **adjacency matrix:** $n \times n$ binary matrix indicating if edge $\{u, v\} \in E$, where $n = |V|$ and $G = (V, E)$. Undirected graph, symmetric matrix.
- Adjacency matrix, directed graph, potentially asymmetric matrix.
- **Incidence matrix.** $n \times m$ matrix of nodes/edges with $n = |V|$ and $m = |E|$.



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$



$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$



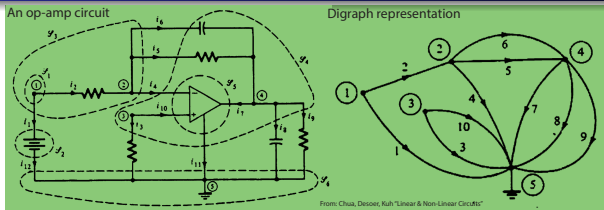
$$\begin{pmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Sparse vs. Dense Graphs & their representations/data structures

- Graphs may be dense (many edges between nodes) or sparse (few edges between nodes).

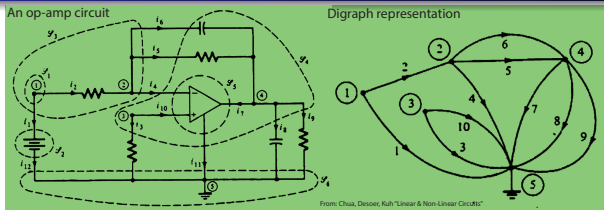
Graphs

Graph vs. Network vs. Circuit: Mostly Synonymous



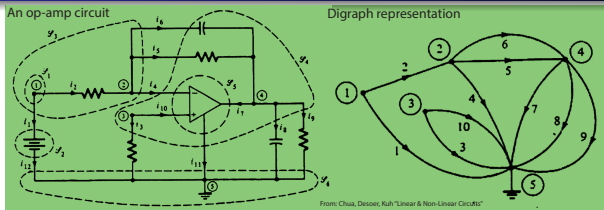
- Represent connectivity & flow: e.g., current in a circuit, data in a computer network, friendship in a social network, core structure for all are nodes (vertices, points) connected by edges (links, arcs) for flow, interaction, or relation.

Graph vs. Network vs. Circuit: Mostly Synonymous



- Represent connectivity & flow: e.g., current in a circuit, data in a computer network, friendship in a social network, core structure for all are nodes (vertices, points) connected by edges (links, arcs) for flow, interaction, or relation.
- Shared mathematical foundation: All one form of graph (nodes + edges) or another, enabling tools an analyses like formal graph theory, matrix algebra (e.g., adjacency, Laplacians), over domains.

Graph vs. Network vs. Circuit: Mostly Synonymous

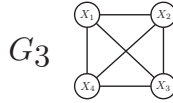
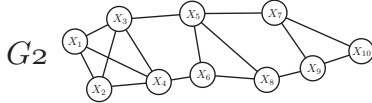
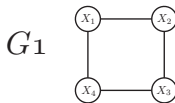


- Represent connectivity & flow: e.g., current in a circuit, data in a computer network, friendship in a social network, core structure for all are nodes (vertices, points) connected by edges (links, arcs) for flow, interaction, or relation.
- Shared mathematical foundation: All one form of graph (nodes + edges) or another, enabling tools and analyses like formal graph theory, matrix algebra (e.g., adjacency, Laplacians), over domains.
- Common laws of interaction: conservation (e.g., Kirchhoff's laws in circuits), propagation (of information), and equilibrium apply across circuits, social influence, metabolic pathways, and routing, all with different physical meanings. e.g., equilibrium could be net node current or voltage around loop zero, stable opinions upon further message passing, blood glucose level maintenance, or load balance.

Some Graph Theory

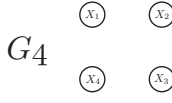
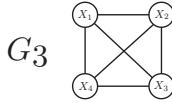
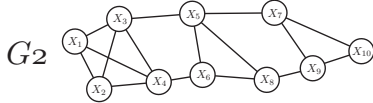
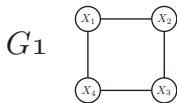
Undirected vs. Directed

- Graphs can be either undirected: Social friendship networks, road intersections without one-way restrictions, collaboration graphs (like co-authorship), undirected graphical models.



Undirected vs. Directed

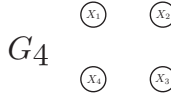
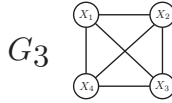
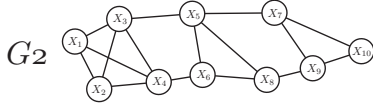
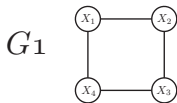
- Graphs can be either undirected: Social friendship networks, road intersections without one-way restrictions, collaboration graphs (like co-authorship), undirected graphical models.



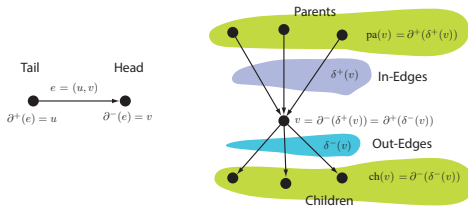
- Undirected graph $G = (V, E)$ where each $e \in E$ has $e = \{u, v\}$, unordered set.

Undirected vs. Directed

- Graphs can be either undirected: Social friendship networks, road intersections without one-way restrictions, collaboration graphs (like co-authorship), undirected graphical models.

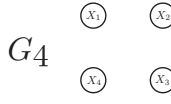
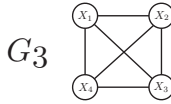
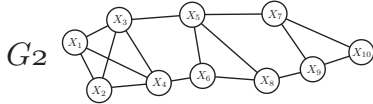
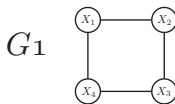


- Undirected graph $G = (V, E)$ where each $e \in E$ has $e = \{u, v\}$, unordered set.
- Or can be directed: Web page links, Twitter follower networks, citation networks, Bayesian networks.

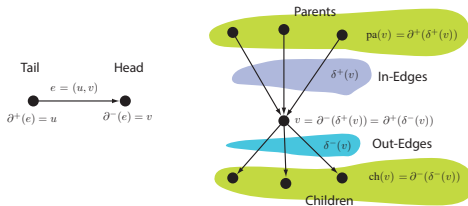


Undirected vs. Directed

- Graphs can be either undirected: Social friendship networks, road intersections without one-way restrictions, collaboration graphs (like co-authorship), undirected graphical models.



- Undirected graph $G = (V, E)$ where each $e \in E$ has $e = \{u, v\}$, unordered set.
- Or can be directed: Web page links, Twitter follower networks, citation networks, Bayesian networks.



- Directed graph (**digraph**) $G = (V, E)$, each $e \in E$ has $e = (u, v)$, from u (**tail**) to v (**head**).

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete set**) if all $u, v \in C$ are connected.

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete** set) if all $u, v \in C$ are connected.
- A set of vertices $C \subseteq V$ is an **independent** set if there exists **no** $u, v \in C$ that are connected (all pairs disconnected).

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete** set) if all $u, v \in C$ are connected.
- A set of vertices $C \subseteq V$ is an **independent** set if there exists **no** $u, v \in C$ that are connected (all pairs disconnected).
- A graph G is **connected** if there exists at least one path between any two vertices.

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete** set) if all $u, v \in C$ are connected.
- A set of vertices $C \subseteq V$ is an **independent** set if there exists **no** $u, v \in C$ that are connected (all pairs disconnected).
- A graph G is **connected** if there exists at least one path between any two vertices.
- A **tree** is a connected graph $G = (V, E)$ with $|V| = n$ nodes and $n - 1$ edges (we'll see other defs. too).

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete** set) if all $u, v \in C$ are connected.
- A set of vertices $C \subseteq V$ is an **independent** set if there exists **no** $u, v \in C$ that are connected (all pairs disconnected).
- A graph G is **connected** if there exists at least one path between any two vertices.
- A **tree** is a connected graph $G = (V, E)$ with $|V| = n$ nodes and $n - 1$ edges (we'll see other defs. too).
- A **forest** is one or more trees.

Paths, Cliques/Complete, Independent Sets, etc.

- Given graph $G = (V, E)$, if $e = \{u, v\} \in E$ we say u and v are **connected** in V .
- If there is a sequence of edges $e_1, e_2, \dots, e_k \in E$ so with $e_1 = \{u, x\}$ and $e_k = \{y, v\}$ for some $x, y \in V$, we say there is a **path** from u to v .
- A **chain** is a sequence of connected vertices $u_1, u_2, \dots, u_\ell$ with $\{u_i, u_{i+1}\} \in E$ for $i \in [\ell - 1]$.
- A set of vertices $C \subseteq V$ is a **clique** (or is a **complete** set) if all $u, v \in C$ are connected.
- A set of vertices $C \subseteq V$ is an **independent** set if there exists **no** $u, v \in C$ that are connected (all pairs disconnected).
- A graph G is **connected** if there exists at least one path between any two vertices.
- A **tree** is a connected graph $G = (V, E)$ with $|V| = n$ nodes and $n - 1$ edges (we'll see other defs. too).
- A **forest** is one or more trees.
- A graph is **planar** if it can be drawn in 2D without any of edges crossing each other.

Cliques and MaxCliques

- In a general graph $G = (V, E)$, we can define:

Cliques and MaxCliques

- In a general graph $G = (V, E)$, we can define:
- **maximum clique**. This is a clique $C \subseteq V$ that is no smaller than any other clique, i.e., $|C| \geq |C'|$ for all $C' \in \mathcal{C}$ where $\mathcal{C}$ is the set of all cliques in G .

Cliques and MaxCliques

- In a general graph $G = (V, E)$, we can define:
- **maximum clique**. This is a clique $C \subseteq V$ that is no smaller than any other clique, i.e., $|C| \geq |C'|$ for all $C' \in \mathcal{C}$ where $\mathcal{C}$ is the set of all cliques in G .
- **maximal clique** or **maxcliques**. This is a clique $C \subseteq V$ that is no longer a clique if we add any node, i.e. C is a clique but $C \cup \{v\}$ is not a clique for any $v \notin C$.

Cliques and MaxCliques

- In a general graph $G = (V, E)$, we can define:
- **maximum clique**. This is a clique $C \subseteq V$ that is no smaller than any other clique, i.e., $|C| \geq |C'|$ for all $C' \in \mathcal{C}$ where $\mathcal{C}$ is the set of all cliques in G .
- **maximal clique** or **maxcliques**. This is a clique $C \subseteq V$ that is no longer a clique if we add any node, i.e. C is a clique but $C \cup \{v\}$ is not a clique for any $v \notin C$.
- The maximum clique problem (finding the largest clique in a general graph is a classic NP-hard problem. Also NP-complete to decide if a graph has a clique of size $\geq k$ (the well-known CLIQUE decision problem).

Cliques and MaxCliques

- In a general graph $G = (V, E)$, we can define:
- **maximum clique**. This is a clique $C \subseteq V$ that is no smaller than any other clique, i.e., $|C| \geq |C'|$ for all $C' \in \mathcal{C}$ where $\mathcal{C}$ is the set of all cliques in G .
- **maximal clique** or **maxcliques**. This is a clique $C \subseteq V$ that is no longer a clique if we add any node, i.e. C is a clique but $C \cup \{v\}$ is not a clique for any $v \notin C$.
- The maximum clique problem (finding the largest clique in a general graph is a classic NP-hard problem. Also NP-complete to decide if a graph has a clique of size $\geq k$ (the well-known CLIQUE decision problem)).
- Also, in general, enumerating all maximal cliques is, in general, hard in total output size, is a #P-complete (sharp-P complete) counting problem. Bron-Kerbosch [1973] algorithm can be used to do this but is worst-case exponential time.

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .

More On Adjacency Matrix and Matrix Powers

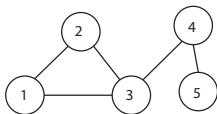
- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .

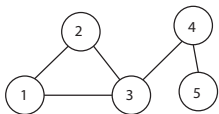
More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .



More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .



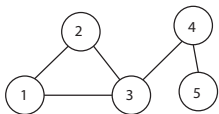
$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 & 0 \\ 1 & 1 & 3 & 0 & 1 \\ 1 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 2 & 3 & 4 & 1 & 1 \\ 3 & 2 & 4 & 1 & 1 \\ 4 & 4 & 2 & 4 & 0 \\ 1 & 1 & 4 & 0 & 2 \\ 1 & 1 & 0 & 2 & 0 \end{bmatrix}$$

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

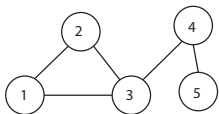
$$A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 & 0 \\ 1 & 1 & 3 & 0 & 1 \\ 1 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 2 & 3 & 4 & 1 & 1 \\ 3 & 2 & 4 & 1 & 1 \\ 4 & 4 & 2 & 4 & 0 \\ 1 & 1 & 4 & 0 & 2 \\ 1 & 1 & 0 & 2 & 0 \end{bmatrix}$$

- Digraph, A is asymmetric, A^k is number of distinct directed length- k paths from $i \rightarrow j$.

More On Adjacency Matrix and Matrix Powers

- Adjacency matrix $A \in \{0, 1\}^{n \times n}$ with $A[i, j] = 1$ iff $\exists$ edge between $i \in V$ and $j \in V$.
- Adjacency matrix is like a similarity, affinity, or connectedness notion, nodes are connected if they are related, or if there is a path of length one between i and j .
- Matrix power $A^k = \underbrace{AAA \dots A}_{k \times}$. Then $A^k[i, j]$ counts the number of paths there are between i and j of length k .



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 1 & 0 \\ 1 & 1 & 3 & 0 & 1 \\ 1 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 2 & 3 & 4 & 1 & 1 \\ 3 & 2 & 4 & 1 & 1 \\ 4 & 4 & 2 & 4 & 0 \\ 1 & 1 & 4 & 0 & 2 \\ 1 & 1 & 0 & 2 & 0 \end{bmatrix}$$

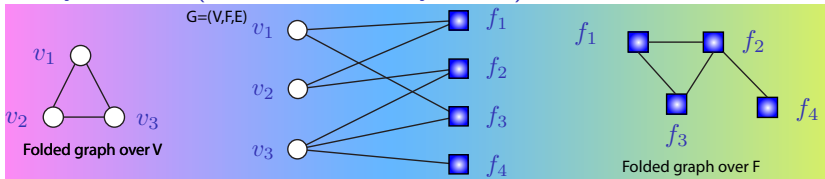
- Digraph, A is asymmetric, A^k is number of distinct directed length- k paths from $i \rightarrow j$.
- With A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have

$$\lim_{k \rightarrow \infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} \quad (2.1)$$

a form of diffusion, or (very roughly) a “geodesic” similarity, or **Katz index**.

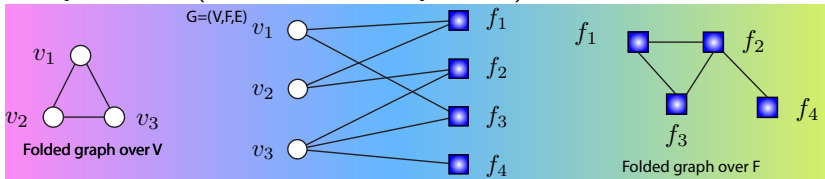
Bipartite Graphs

- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.



Bipartite Graphs

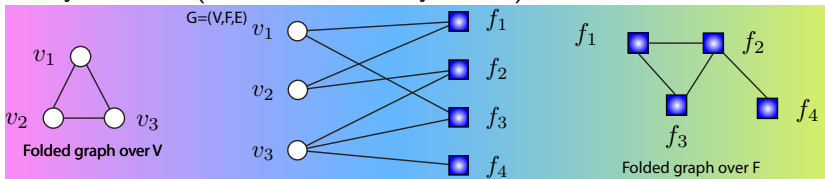
- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.



- Apps: recommendation systems (users-items, users-movies), matching problems (e.g., job applicants and positions), biological networks (e.g., genes and diseases), factor graphs.

Bipartite Graphs

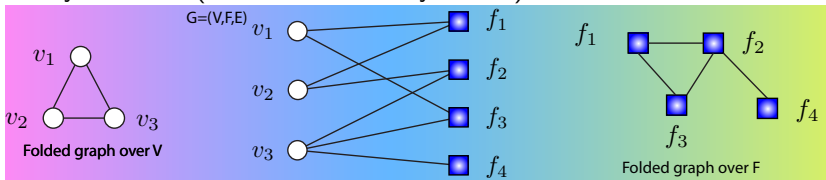
- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.



- Apps: recommendation systems (users-items, users-movies), matching problems (e.g., job applicants and positions), biological networks (e.g., genes and diseases), factor graphs.
- Any graph $G = (V, E)$ expressible by a bipartite graph $G' = (V, E, F)$ where V is on the left, G 's edges E are now nodes on the right, and for each $e = \{u, v\} \in E$ we have an edge in G' from $u \in V$ to node $e \in E$ and from $v \in V$ to node $e \in E$.

Bipartite Graphs

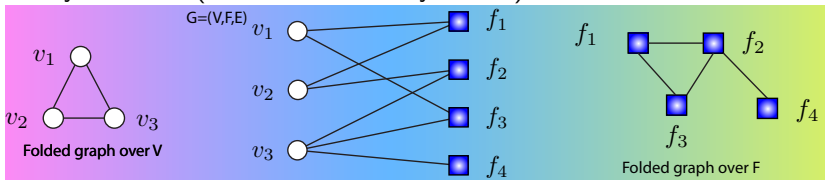
- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.



- Apps: recommendation systems (users-items, users-movies), matching problems (e.g., job applicants and positions), biological networks (e.g., genes and diseases), factor graphs.
- Any graph $G = (V, E)$ expressible by a bipartite graph $G' = (V, E, F)$ where V is on the left, G' 's edges E are now nodes on the right, and for each $e = \{u, v\} \in E$ we have an edge in G' from $u \in V$ to node $e \in E$ and from $v \in V$ to node $e \in E$.
- Generalizing: Multipartite graphs, nodes partitioned into (disjoint) sets $V = V_1 \cup V_2 \cup \dots \cup V_k$ where edges can exist only between but not within sets.

Bipartite Graphs

- Bipartite graph: nodes partitioned into two (disjoint) sets (V on the left, F on the right), edges exist only between (but not within any of the) sets.



- Apps: recommendation systems (users-items, users-movies), matching problems (e.g., job applicants and positions), biological networks (e.g., genes and diseases), factor graphs.
- Any graph $G = (V, E)$ expressible by a bipartite graph $G' = (V, E, F)$ where V is on the left, G' 's edges E are now nodes on the right, and for each $e = \{u, v\} \in E$ we have an edge in G' from $u \in V$ to node $e \in E$ and from $v \in V$ to node $e \in E$.
- Generalizing: Multipartite graphs, nodes partitioned into (disjoint) sets $V = V_1 \cup V_2 \cup \dots \cup V_k$ where edges can exist only between but not within sets.
- Apps: heterogeneous info. networks (e.g., authors, papers, venues), multiple-entity-type knowledge graphs, supply chain networks (products, suppliers, consumers), etc.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Definition 2.3.1 (hypergraph)

A *hypergraph* $H = (V, E)$ is a set of vertices V and a collection of hyperedges E , where each element $e \in E$ is a subset of V , so $\forall e \in E, e \subseteq V$.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Definition 2.3.1 (hypergraph)

A *hypergraph* $H = (V, E)$ is a set of vertices V and a collection of hyperedges E , where each element $e \in E$ is a subset of V , so $\forall e \in E, e \subseteq V$.

- In a graph size of e is always two, but in a hypergraph, $|e| \geq 2$.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Definition 2.3.1 (hypergraph)

A *hypergraph* $H = (V, E)$ is a set of vertices V and a collection of hyperedges E , where each element $e \in E$ is a subset of V , so $\forall e \in E, e \subseteq V$.

- In a graph size of e is always two, but in a hypergraph, $|e| \geq 2$.
- Since a graph, $|e| = 2$, a graph is a (restricted) hypergraph, but not vice verse.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Definition 2.3.1 (hypergraph)

A *hypergraph* $H = (V, E)$ is a set of vertices V and a collection of hyperedges E , where each element $e \in E$ is a subset of V , so $\forall e \in E, e \subseteq V$.

- In a graph size of e is always two, but in a hypergraph, $|e| \geq 2$.
- Since a graph, $|e| = 2$, a graph is a (restricted) hypergraph, but not vice verse.
- A set system is a set and a set of subsets of that set.

Hypergraphs

- Recall, a graph $G = (V, E)$ is a set of nodes V and edges E where every $(s, t) = e \in E$ is only two nodes.
- A hypergraph is a generalization of a graph.

Definition 2.3.1 (hypergraph)

A *hypergraph* $H = (V, E)$ is a set of vertices V and a collection of hyperedges E , where each element $e \in E$ is a subset of V , so $\forall e \in E, e \subseteq V$.

- In a graph size of e is always two, but in a hypergraph, $|e| \geq 2$.
- Since a graph, $|e| = 2$, a graph is a (restricted) hypergraph, but not vice versa.
- A set system is a set and a set of subsets of that set.
- Thus, a **hypergraph** is a set system (V, E) where every $e \in E$ can consist of any number of nodes. I.e., we might have $\{v_1, v_2, \dots, v_{k_e}\} = e \in E(G)$ for a hypergraph.

Hypergraphs and bipartite graphs

- Hypergraphs can be represented by a bipartite $G = (V, F, E)$ graphs where V is a set of left-nodes, F is a set of right nodes, and E is a set of size-two edges.

Hypergraphs and bipartite graphs

- Hypergraphs can be represented by a bipartite $G = (V, F, E)$ graphs where V is a set of left-nodes, F is a set of right nodes, and E is a set of size-two edges.
- Right nodes are hyperedges in the hypergraphs.

Hypergraphs and bipartite graphs

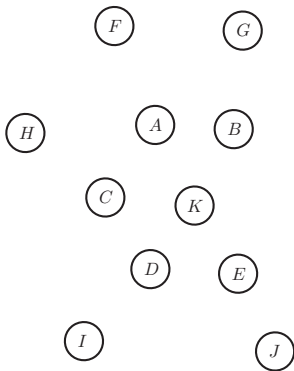
- Hypergraphs can be represented by a bipartite $G = (V, F, E)$ graphs where V is a set of left-nodes, F is a set of right nodes, and E is a set of size-two edges.
- Right nodes are hyperedges in the hypergraphs.
- That is, we have a restriction that $\forall e \in E$ with $e = (v, f)$ we must have $v \in V$ and $f \in F$.

Hypergraphs and bipartite graphs

- Hypergraphs can be represented by a bipartite $G = (V, F, E)$ graphs where V is a set of left-nodes, F is a set of right nodes, and E is a set of size-two edges.
- Right nodes are hyperedges in the hypergraphs.
- That is, we have a restriction that $\forall e \in E$ with $e = (v, f)$ we must have $v \in V$ and $f \in F$.
- Next slide shows an example.

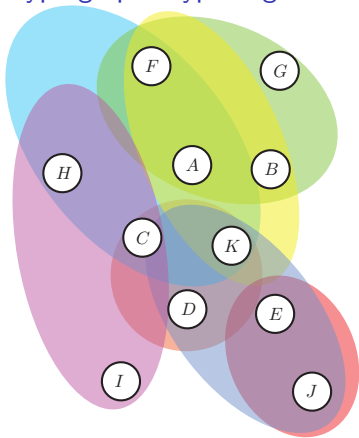
Drawing/Visualizing Hypergraphs & Bipartite graphs

- A set of vertices, normally edges connect two nodes.



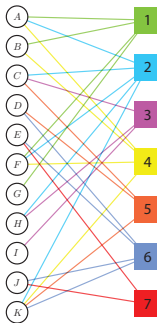
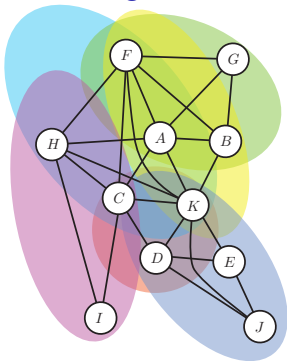
Drawing/Visualizing Hypergraphs & Bipartite graphs

- A set of vertices, normally edges connect two nodes.
- Hypergraph: hyperedges are shaded regions, each region a vertex cluster



Drawing/Visualizing Hypergraphs & Bipartite graphs

- A set of vertices, normally edges connect two nodes.
- Hypergraph: hyperedges are shaded regions, each region a vertex cluster
- Shaded regions are cluster edge cover of “conformal” (TBD) graph.



Graph of a hypergraph, conformal, and acyclic

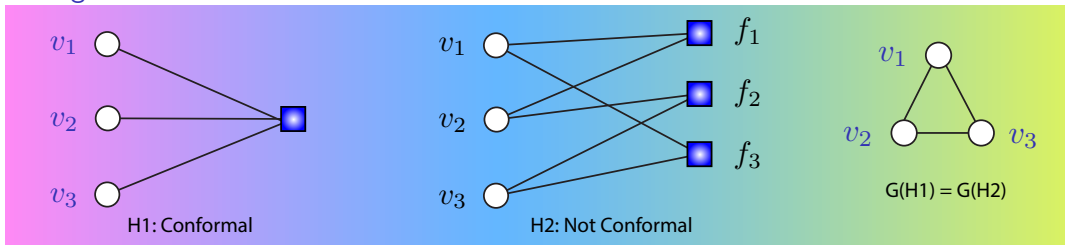
- Let $H = (V, E)$ be a hypergraph with vertex set V and edge set E .

Graph of a hypergraph, conformal, and acyclic

- Let $H = (V, E)$ be a hypergraph with vertex set V and edge set E .
- The **graph of a hypergraph** $G(H)$ is a graph $G(H) = (V, E')$ where E' is a set of vertex pairs, and where there is an edge in E' for every pair of nodes that are in the same hyperedge.

Graph of a hypergraph, conformal, and acyclic

- Let $H = (V, E)$ be a hypergraph with vertex set V and edge set E .
- The **graph of a hypergraph** $G(H)$ is a graph $G(H) = (V, E')$ where E' is a set of vertex pairs, and where there is an edge in E' for every pair of nodes that are in the same hyperedge.
- A hypergraph H is **conformal** (to the graph $G(H)$) if every clique of $G(H)$ is contained in an edge of H .



Graph of a hypergraph, conformal, and acyclic

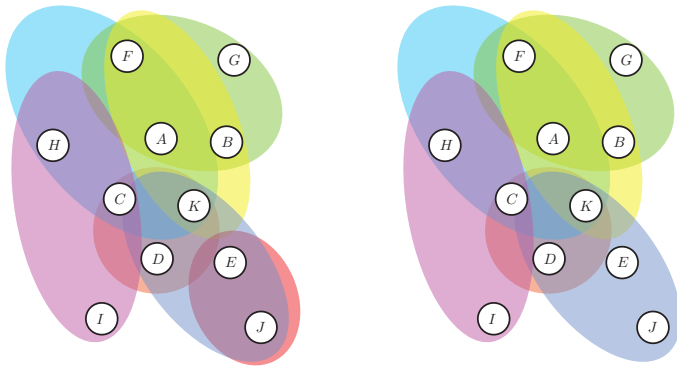
- Let $H = (V, E)$ be a hypergraph with vertex set V and edge set E .
- The **graph of a hypergraph** $G(H)$ is a graph $G(H) = (V, E')$ where E' is a set of vertex pairs, and where there is an edge in E' for every pair of nodes that are in the same hyperedge.
- A hypergraph H is **conformal** (to the graph $G(H)$) if every clique of $G(H)$ is contained in an edge of H .
- Note, hypergraph H with hyperedges associated only with graph edges of $G(H)$ is not conformal (since cliques of $G(H)$ might not be fully contained in an edge of H).

Hypergraph vs. Reduced Hypergraph

- A hypergraph is **reduced** if no edge is a subset of another edge.

Hypergraph vs. Reduced Hypergraph

- A hypergraph is **reduced** if no edge is a subset of another edge.
- Hypergraph (as shaded regions) on left, reduced hypergraph on the right (i.e., hyper edge $\{E, J\} \subset \{E, J, D, K\}$ is removed).

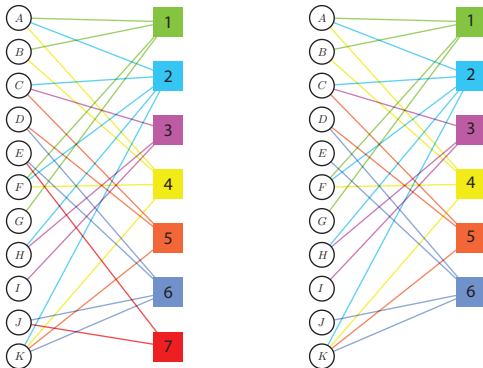


Hypergraph vs. Reduced Hypergraph: Bipartite Form

- A hypergraph is **reduced** if no edge is a subset of another edge.

Hypergraph vs. Reduced Hypergraph: Bipartite Form

- A hypergraph is **reduced** if no edge is a subset of another edge.
- Hypergraph (as bipartite graph) on left, reduced hypergraph on the right (edge $\{E,J\}$ removed).



Connected Components of a Graph

- **Connected component of a graph:** a piece of graph that is connected (e.g., consider two moons k -NN graph with two components). By “piece”, we mean a maximal connected vertex induced subgraph.

Connected Components of a Graph

- **Connected component of a graph**: a piece of graph that is connected (e.g., consider two moons k -NN graph with two components). By “piece”, we mean a maximal connected vertex induced subgraph.
- A connected graph has one connected components.

Connected Components of a Graph

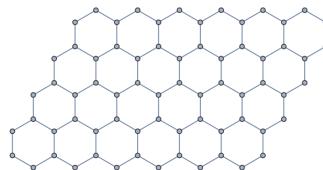
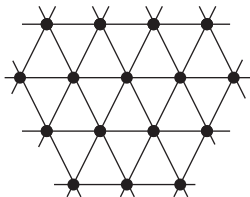
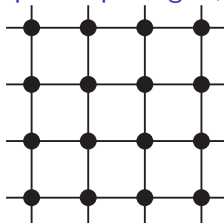
- **Connected component of a graph**: a piece of graph that is connected (e.g., consider two moons k -NN graph with two components). By “piece”, we mean a maximal connected vertex induced subgraph.
- A connected graph has one connected components.
- A tree has one connected component, while a forest can have more.

Grids

- A grid graph (or lattice or mesh) is a graph whose vertices and edges form a regular repeating tiling in 2D.

Grids

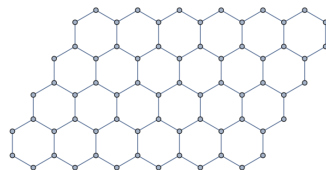
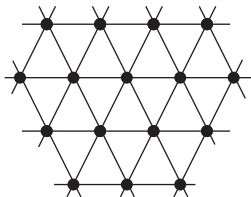
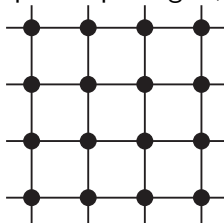
- A grid graph (or lattice or mesh) is a graph whose vertices and edges form a regular repeating tiling in 2D.
- Examples: square grid, triangular grid, hexagonal grid.



From: Mathematica

Grids

- A grid graph (or lattice or mesh) is a graph whose vertices and edges form a regular repeating tiling in 2D.
- Examples: square grid, triangular grid, hexagonal grid.

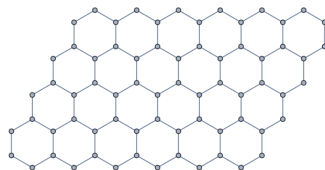
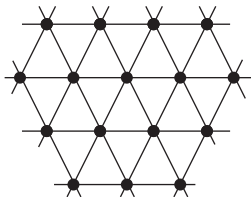
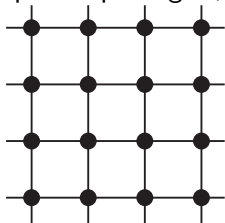


From: Mathematica

- Used in image processing, path planning, geometric modeling, etc.

Grids

- A grid graph (or lattice or mesh) is a graph whose vertices and edges form a regular repeating tiling in 2D.
- Examples: square grid, triangular grid, hexagonal grid.

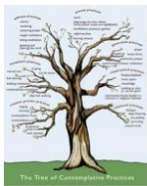


From: Mathematica

- Used in image processing, path planning, geometric modeling, etc.
- These are common in Markov random fields (a type of undirected graphical model).

More Trees

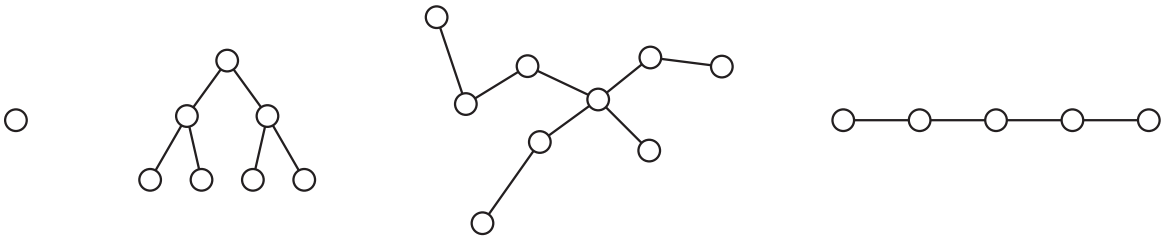
Trees



Trees defined

Definition 2.4.1

A graph $G = (V, E)$ is a *forest* if it is the case that for all $u, v \in V$, there is no more than one path that connects u to v in G . Given a forest G , if for all $u, v \in G$ there is a unique path connecting u and v , then it is called a *connected forest* or just simply a *tree*.



Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- *G is connected and has no cycles*

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- *G is connected and has no cycles*
- *G has $n - 1$ edges and has no cycles,*

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- G is connected and has no cycles
- G has $n - 1$ edges and has no cycles,
- G is connected and contains exactly $n - 1$ edges,

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- *G is connected and has no cycles*
- *G has $n - 1$ edges and has no cycles,*
- *G is connected and contains exactly $n - 1$ edges,*
- *G has no cycles. Exactly one cycle created if edge added to G .*

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- G is connected and has no cycles
- G has $n - 1$ edges and has no cycles,
- G is connected and contains exactly $n - 1$ edges,
- G has no cycles. Exactly one cycle created if edge added to G .
- G is connected, and if any edge is removed, the remaining graph is not connected,

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- G is connected and has no cycles
- G has $n - 1$ edges and has no cycles,
- G is connected and contains exactly $n - 1$ edges,
- G has no cycles. Exactly one cycle created if edge added to G .
- G is connected, and if any edge is removed, the remaining graph is not connected,
- Every pair of vertices of G is connected by one unique path.

Trees defined in many ways

Theorem 2.4.2 (Trees, Berge)

Let $G = (V, E)$ be an undirected graph with $|V| = n > 2$. Then each of the following properties are equivalent and each can be used to define when G is a tree:

- G is connected and has no cycles
- G has $n - 1$ edges and has no cycles,
- G is connected and contains exactly $n - 1$ edges,
- G has no cycles. Exactly one cycle created if edge added to G .
- G is connected, and if any edge is removed, the remaining graph is not connected,
- Every pair of vertices of G is connected by one unique path.
- G can be generated as follows: Start with v , repeatedly choose next vertex, and connect it with edge to exactly one previous vertex.

Node Statistics

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

- If graph is weighted with $w : E \rightarrow \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$.
 $w(u, v) = 0$ means u is not connected to v , we have a weighted similarity, affinity, or connection strength matrix W :

$$W_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}, \quad (\text{unweighted case } w(i, j) \in \{0, 1\}). \quad (2.3)$$

Notation A , W common in spectral graph theory, diffusion and kernel methods, and GNNs.

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

- If graph is weighted with $w : E \rightarrow \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$.
 $w(u, v) = 0$ means u is not connected to v , we have a weighted similarity, affinity, or connection strength matrix W :

$$W_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}, \quad (\text{unweighted case } w(i, j) \in \{0, 1\}). \quad (2.3)$$

Notation A , W common in spectral graph theory, diffusion and kernel methods, and GNNs.

- degree of a vertex** $v \in V$, $d_v = \deg(v) = |\{u \in V \mid \{u, v\} \in E\}|$.

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

- If graph is weighted with $w : E \rightarrow \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$.
 $w(u, v) = 0$ means u is not connected to v , we have a weighted similarity, affinity, or connection strength matrix W :

$$W_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}, \quad (\text{unweighted case } w(i, j) \in \{0, 1\}). \quad (2.3)$$

Notation A , W common in spectral graph theory, diffusion and kernel methods, and GNNs.

- degree of a vertex** $v \in V$, $d_v = \deg(v) = |\{u \in V \mid \{u, v\} \in E\}|$.
- degree of a vertex in a weighted graph** $d_i = \sum_{j=1}^n w_{ij}$, (matrix row-sum or col-sum i).

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

- If graph is weighted with $w : E \rightarrow \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$.
 $w(u, v) = 0$ means u is not connected to v , we have a weighted similarity, affinity, or connection strength matrix W :

$$W_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}, \quad (\text{unweighted case } w(i, j) \in \{0, 1\}). \quad (2.3)$$

Notation A , W common in spectral graph theory, diffusion and kernel methods, and GNNs.

- degree of a vertex** $v \in V$, $d_v = \deg(v) = |\{u \in V \mid \{u, v\} \in E\}|$.
- degree of a vertex** in a weighted graph $d_i = \sum_{j=1}^n w_{ij}$, (matrix row-sum or col-sum i).
- Either case, can define **degree matrix** $D = \text{diag}(d_1, d_2, \dots, d_n)$. D gives “weighted” degree (although for 0/1-valued weights, counts the number of edges incident to v_i).

Nodes and Degrees

- Graph $G = (V, E, w)$ where $|V| = n$ and $V = \{1, \dots, n\}$ with adjacency matrix A :

$$A_{ij} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases} \quad (2.2)$$

- If graph is weighted with $w : E \rightarrow \mathbb{R}_+$ is weight function so for $e \in E$, $w(e) \in \mathbb{R}_+$.
 $w(u, v) = 0$ means u is not connected to v , we have a weighted similarity, affinity, or connection strength matrix W :

$$W_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}, \quad (\text{unweighted case } w(i, j) \in \{0, 1\}). \quad (2.3)$$

Notation A , W common in spectral graph theory, diffusion and kernel methods, and GNNs.

- degree of a vertex** $v \in V$, $d_v = \deg(v) = |\{u \in V \mid \{u, v\} \in E\}|$.
- degree of a vertex** in a weighted graph $d_i = \sum_{j=1}^n w_{ij}$, (matrix row-sum or col-sum i).
- Either case, can define **degree matrix** $D = \text{diag}(d_1, d_2, \dots, d_n)$. D gives “weighted” degree (although for 0/1-valued weights, counts the number of edges incident to v_i).
- Both undirected and directed graphs, in the directed case A and W not symmetric.

In/Out Degrees

- We used “ $W_{ij} = w(i, j)$ if $(i, j) \in E$ ” notation (i, j) deliberately. We think of an undirected graph as having edges both (i, j) and (j, i) while in directed case not necessarily so.

In/Out Degrees

- We used “ $W_{ij} = w(i, j)$ if $(i, j) \in E$ ” notation (i, j) deliberately. We think of an undirected graph as having edges both (i, j) and (j, i) while in directed case not necessarily so.
- (i, j) is from i to j .

In/Out Degrees

- We used “ $W_{ij} = w(i, j)$ if $(i, j) \in E$ ” notation (i, j) deliberately. We think of an undirected graph as having edges both (i, j) and (j, i) while in directed case not necessarily so.
- (i, j) is from i to j .
- Directed graph **out-degree** of a vertex: $d_i^{\text{out}} = \sum_{j=1}^n w_{ij}$ (row sum).

In/Out Degrees

- We used “ $W_{ij} = w(i, j)$ if $(i, j) \in E$ ” notation (i, j) deliberately. We think of an undirected graph as having edges both (i, j) and (j, i) while in directed case not necessarily so.
- (i, j) is from i to j .
- Directed graph **out-degree** of a vertex: $d_i^{\text{out}} = \sum_{j=1}^n w_{ij}$ (row sum).
- Directed graph **in-degree** of a vertex: $d_j^{\text{in}} = \sum_{i=1}^n w_{ij}$ (col-sum).

Other Core Node-Level Statistics: Centrality

- **Betweenness Centrality**, for all $v \in V$:

$$c_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

I.e., fraction of shortest paths between all node pairs (s, t) that pass through v , where $\sigma_{st}(v)$ is the number of such paths and $\sigma_{st} = \sum_v \sigma_{st}(v)$.

Other Core Node-Level Statistics: Centrality

- **Betweenness Centrality**, for all $v \in V$:

$$c_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

I.e., fraction of shortest paths between all node pairs (s, t) that pass through v , where $\sigma_{st}(v)$ is the number of such paths and $\sigma_{st} = \sum_v \sigma_{st}(v)$.

- **Closeness Centrality**, for all $v \in V$:

$$c_C(v) = \frac{1}{\sum_{u \in V} d(v, u)}$$

I.e., inverse of the total distance from v to all other nodes, where $d(v, u)$ is length of shortest path from v to u in graph.

Other Core Node-Level Statistics: Eigen Centrality

- Eigenvector Centrality

$$c(v) = \frac{1}{\lambda} \sum_{u \in \mathcal{N}(v)} c_u$$

Node importance where connections to high-scoring neighbors contribute more. Typically computed with power iteration.

Algorithm 2: Power Iteration for Eigenvector Centrality

Input : Adjacency matrix $A \in \mathbb{R}^{n \times n}$, tolerance $\varepsilon > 0$, max iterations T

Output: Centrality vector $c \propto$ leading eigenvector of A

```
1 Initialize  $c^{(0)} \leftarrow \mathbf{1} / \|\mathbf{1}\|_2$  ;
2 for  $t \leftarrow 1$  to  $T$  do
3    $c^{(t)} \leftarrow A c^{(t-1)}$  ;
4    $c^{(t)} \leftarrow c^{(t)} / \|c^{(t)}\|_2$  ;
5   if  $\|c^{(t)} - c^{(t-1)}\|_2 < \varepsilon$  then
6     break ;
7 return  $c^{(t)}$ 
```

Other Core Node-Level Statistics: Eigen Centrality

- Eigenvector Centrality

$$c(v) = \frac{1}{\lambda} \sum_{u \in \mathcal{N}(v)} c_u \Rightarrow \lambda c = A c \Rightarrow \text{Eigenvector/value problem.}$$

Node importance where connections to high-scoring neighbors contribute more. Typically computed with power iteration.

Algorithm 3: Power Iteration for Eigenvector Centrality

Input : Adjacency matrix $A \in \mathbb{R}^{n \times n}$, tolerance $\varepsilon > 0$, max iterations T

Output: Centrality vector $c \propto$ leading eigenvector of A

```
1 Initialize  $c^{(0)} \leftarrow \mathbf{1} / \|\mathbf{1}\|_2$  ;
2 for  $t \leftarrow 1$  to  $T$  do
3    $c^{(t)} \leftarrow A c^{(t-1)}$  ;
4    $c^{(t)} \leftarrow c^{(t)} / \|c^{(t)}\|_2$  ;
5   if  $\|c^{(t)} - c^{(t-1)}\|_2 < \varepsilon$  then
6     break ;
7 return  $c^{(t)}$ 
```

Other Core Node-Level Statistics: Page Rank

- PageRank

$$c(v) = \alpha \sum_{u \in \mathcal{N}(v)} \frac{c(u)}{\deg(u)} + (1 - \alpha) \frac{1}{|V|} \Rightarrow c = \alpha AD^{-1}c + (1 - \alpha)\mathbf{1}/n$$

random-walk-based importance (similar to Eigen centrality) but with restart probability $1 - \alpha$, where D is diagonal matrix $D_{ii} = \deg(i)$, A adjacency matrix (directed/undirected).

Other Core Node-Level Statistics: Page Rank

- PageRank

$$c(v) = \alpha \sum_{u \in \mathcal{N}(v)} \frac{c(u)}{\deg(u)} + (1 - \alpha) \frac{1}{|V|} \Rightarrow c = \alpha A D^{-1} c + (1 - \alpha) \mathbf{1}/n$$

random-walk-based importance (similar to Eigen centrality) but with restart probability $1 - \alpha$, where D is diagonal matrix $D_{ii} = \deg(i)$, A adjacency matrix (directed/undirected).

- Again, Eigen equation solvable with power iteration:

Algorithm 5: An Example of Power Iteration for a PageRank Variant

Input : Adjacency matrix $A \in \mathbb{R}^{n \times n}$, damping factor $\alpha \in (0, 1)$, tolerance ε , max iterations T

Output: PageRank vector $c \in \mathbb{R}^n$

```
1 Compute degree vector  $d_i \leftarrow \sum_j A_{ij}$  ;
2 Initialize  $c^{(0)} \leftarrow \mathbf{1}/n$  for  $t \leftarrow 1$  to  $T$  do
3   for  $i \leftarrow 1$  to  $n$  do
4      $c_i^{(t)} \leftarrow \alpha \sum_{j=1}^n \frac{A_{ji}}{d_j} c_j^{(t-1)} + (1 - \alpha)/n$ 
5   if  $\|c^{(t)} - c^{(t-1)}\|_1 < \varepsilon$  then
6     break
7 return  $c^{(t)}$ 
```


Similarity Graphs

- fully connected: E.g., $s_{ij} = \exp(-\|x^{(i)} - x^{(j)}\|/(2\sigma^2)) > 0$ so all edges are present. $G = (V, E)$ is a big clique with $n = |V|$ nodes.

Similarity Graphs

- fully connected: E.g., $s_{ij} = \exp(-\|x^{(i)} - x^{(j)}\|/(2\sigma^2)) > 0$ so all edges are present. $G = (V, E)$ is a big clique with $n = |V|$ nodes. σ is the width of the kernel, as σ gets small, edges between distant nodes have very small weight!

Similarity Graphs

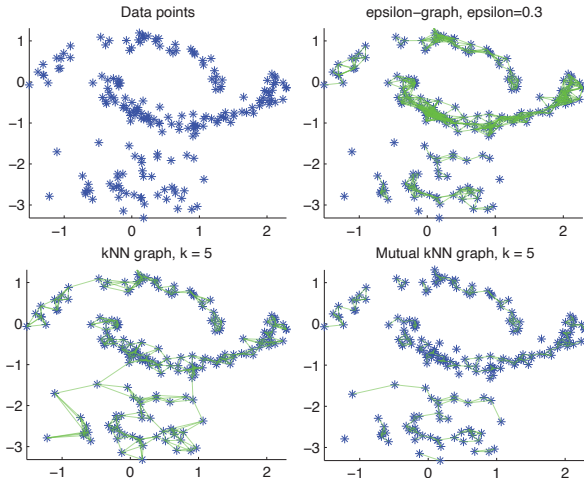
- fully connected: E.g., $s_{ij} = \exp(-\|x^{(i)} - x^{(j)}\|/(2\sigma^2)) > 0$ so all edges are present. $G = (V, E)$ is a big clique with $n = |V|$ nodes. σ is the width of the kernel, as σ gets small, edges between distant nodes have very small weight!
- k -NN graph. Each node is only connected to its k nearest neighbors. Two ways to make adjacency matrix symmetric: $W \leftarrow \max(W, W^\top)$ leading to k -nearest neighbor graph, or $W \leftarrow \min(W, W^\top)$ (the mutual k -nearest neighbor graph).

Similarity Graphs

- fully connected: E.g., $s_{ij} = \exp(-\|x^{(i)} - x^{(j)}\|/(2\sigma^2)) > 0$ so all edges are present. $G = (V, E)$ is a big clique with $n = |V|$ nodes. σ is the width of the kernel, as σ gets small, edges between distant nodes have very small weight!
- k -NN graph. Each node is only connected to its k nearest neighbors. Two ways to make adjacency matrix symmetric: $W \leftarrow \max(W, W^\top)$ leading to k -nearest neighbor graph, or $W \leftarrow \min(W, W^\top)$ (the mutual k -nearest neighbor graph).
- ϵ -neighborhood graph: connect points whose distance is $< \epsilon$. Often these are unweighted.

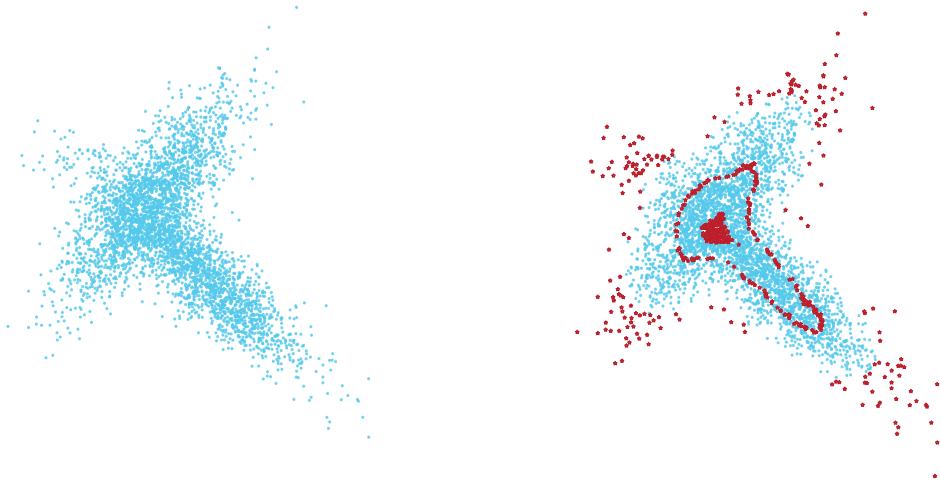
Similarity Graph Examples

- Which similarity graph chosen can make a big difference. Example, sample of two-moons with some additional Gaussian points below.



Eigen Centrality Example: Most, Least, and Mid

Left: 4000 data points in 2D. Right: most, least, and mid-level centrality values are marked from a k -NN graph with $k = 100$ using RBF affinity $s(i, j) = \exp(-\|x_i - x_j\|_2 / \sigma^2)$ with σ^2 being the mean distance.



Still Other Core Node-Level Statistics

- **Local Clustering Coefficient:**

$$c_v = \frac{2 \cdot T(v)}{\deg(v)(\deg(v) - 1)} = \frac{T(v)}{\binom{\deg(v)}{2}}$$

Proportion of a node's neighbors that are connected to each other; $T(v)$ is the number of triangles that include v , i.e., $T(v) = |\{\{u, w\} \subset \mathcal{N}(v) \mid (u, w) \in E\}|$, and again $\mathcal{N}(v)$ are v 's neighbors.

- **K-Core Number:** The largest k such that node v is in the k -core (a maximal v -containing subgraph where all nodes have degree $\geq k$). “buried” centrality.
- **Eccentricity:**

$$\varepsilon(v) = \max_{u \in V} d(v, u)$$

Distance from v to the farthest reachable node.

- There are many others.

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.
- Given degree matrix D (i.e., a diagonal matrix with entries $d_i = \sum_{j=1}^n w_{ij}$) and symmetric adjacency matrix W , define the unnormalized graph Laplacian matrix as

$$L = D - W \tag{2.4}$$

Graph Laplacian

- An amazing amount of information about a graph can be expressed by properties of its Laplacian matrix.
- Given degree matrix D (i.e., a diagonal matrix with entries $d_i = \sum_{j=1}^n w_{ij}$) and symmetric adjacency matrix W , define the unnormalized graph Laplacian matrix as

$$L = D - W \tag{2.4}$$

- Note $L_{ii} = d_{ii} - w_{ii} = \sum_{j=1}^n w_{ij} - w_{ii} = \sum_{j \neq i} w_{ij}$, so diagonal of W is irrelevant to L (e.g., self-edges & self-similarities don't matter).

Properties of the graph Laplacian

Proposition 2.5.1 (Properties of the unnormalized graph Laplacian L)

- ① For every vector $z \in \mathbb{R}^n$, we have, for $L = D - W$,

$$z^\top L z = \frac{1}{2} \sum_{i,j=1}^n w_{ij} (z_i - z_j)^2 \geq 0 \quad (2.5)$$

- ② L is symmetric and positive semi-definite.
- ③ The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant one vector $\mathbf{1} = \mathbf{1}_V \in \mathbb{R}^n$.
- ④ L has n non-negative real valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.

Properties of the graph Laplacian

Proposition 2.5.1 (Properties of the unnormalized graph Laplacian L)

- ① For every vector $z \in \mathbb{R}^n$, we have, for $L = D - W$,

$$z^\top Lz = \frac{1}{2} \sum_{i,j=1}^n w_{ij}(z_i - z_j)^2 \geq 0 \quad (2.5)$$

- ② L is symmetric and positive semi-definite.
- ③ The smallest eigenvalue of L is 0, and the corresponding eigenvector is the constant one vector $\mathbf{1} = \mathbf{1}_V \in \mathbb{R}^n$.
- ④ L has n non-negative real valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.

$$\begin{aligned} z^\top Lz &= z^\top Dz - z^\top Wz = \sum_{i=1}^n d_i z_i^2 - \sum_{i,j=1}^n z_i z_j w_{ij} \\ &= \frac{1}{2} \left(\sum_{i=1}^n d_i z_i^2 - 2 \sum_{i,j=1}^n z_i z_j w_{ij} + \sum_{j=1}^n d_j z_j^2 \right) = \frac{1}{2} \sum_{i,j=1}^n w_{ij}(z_i - z_j)^2 \end{aligned} \quad (2.6)$$

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length- k paths $i \rightarrow j$, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have “geodesic” variant:

$$\lim_{k \rightarrow \infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k \rightarrow \infty} \sum_k X^k = (I - X)^{-1} \quad (2.7)$$

a “geodesic” or “transitive” similarity or relationship strength.

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length- k paths $i \rightarrow j$, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have “geodesic” variant:

$$\lim_{k \rightarrow \infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k \rightarrow \infty} \sum_k X^k = (I - X)^{-1} \quad (2.7)$$

a “geodesic” or “transitive” similarity or relationship strength.

- Consider a graph Laplacian form of this (a diffusion kernel) with $\lambda > 0$

$$K = (D - W + \lambda I)^{-1} = (L + \lambda I)^{-1} \quad (2.8)$$

This is the inverse of symmetric, positive-definite matrix w. non-positive off-diagonals (P.D. M-matrix), can be shown to have non-negative entries <https://en.wikipedia.org/wiki/M-matrix>

Diffusion Kernel Based on Laplacian

- Recall A^k is number of distinct length- k paths $i \rightarrow j$, and diffusion, i.e., A sparse non-negative similarity matrix, then with $\beta < 1/\lambda_{\max}(A)$ we have “geodesic” variant:

$$\lim_{k \rightarrow \infty} \sum_k \beta^k A^k = (I - \beta A)^{-1} = \lim_{k \rightarrow \infty} \sum_k X^k = (I - X)^{-1} \quad (2.7)$$

a “geodesic” or “transitive” similarity or relationship strength.

- Consider a graph Laplacian form of this (a diffusion kernel) with $\lambda > 0$

$$K = (D - W + \lambda I)^{-1} = (L + \lambda I)^{-1} \quad (2.8)$$

This is the inverse of symmetric, positive-definite matrix w. non-positive off-diagonals (P.D. M-matrix), can be shown to have non-negative entries <https://en.wikipedia.org/wiki/M-matrix>

- We can equate the above, by setting $(I - X) = L + \lambda I$, to get:

$$X = (1 - \lambda)I - L = W - D + (1 - \lambda)I$$

- W : weighted adjacency (direct affinities), off-diagonal entries same as W .
- $-D$: discounts/penalizes high-degree nodes (hub suppression, deflation, penalize popularity)
- $(1 - \lambda)I$: regularization, identity bias, controls self-weight, further deflation, as λ increases decays more over path length.

Edge Statistics

Edge Stats

- There exist **edge-based statistics** too, useful for thinks like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:

Edge Stats

- There exist **edge-based statistics** too, useful for things like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:
- **Neighborhood Overlap**, useful in link prediction.

$$S[i, j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \quad (2.9)$$

Edge Stats

- There exist **edge-based statistics** too, useful for things like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:
- **Neighborhood Overlap**, useful in link prediction.

$$S[i, j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \quad (2.9)$$

- **Jaccard similarity**, normalized neighborhood overlap:

$$J[i, j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \quad (2.10)$$

Edge Stats

- There exist **edge-based statistics** too, useful for things like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:
- **Neighborhood Overlap**, useful in link prediction.

$$S[i, j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \quad (2.9)$$

- **Jaccard similarity**, normalized neighborhood overlap:

$$J[i, j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \quad (2.10)$$

- **Adamic-Adar index**, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$AA[i, j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \quad (2.11)$$

Edge Stats

- There exist **edge-based statistics** too, useful for things like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:
- **Neighborhood Overlap**, useful in link prediction.

$$S[i, j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \quad (2.9)$$

- **Jaccard similarity**, normalized neighborhood overlap:

$$J[i, j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \quad (2.10)$$

- **Adamic-Adar index**, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$AA[i, j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \quad (2.11)$$

- **Preferential Attachment**, aspects of "rich get richer" in networks, e.g., in link prediction, to estimate the likelihood of new edges forming between high-degree nodes:

$$PA[i, j] = \deg(i) \cdot \deg(j) \quad (2.12)$$

Edge Stats

- There exist **edge-based statistics** too, useful for things like **link prediction**, **edge classification**, and/or **edge weighting**. Here are a few important ones:
- **Neighborhood Overlap**, useful in link prediction.

$$S[i, j] = |\mathcal{N}(i) \cap \mathcal{N}(j)| \quad (2.9)$$

- **Jaccard similarity**, normalized neighborhood overlap:

$$J[i, j] = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|} \quad (2.10)$$

- **Adamic-Adar index**, common neighbors weighted inversely by their popularity. E.g., in link prediction to suggest new friends based on shared **rare** connections.

$$AA[i, j] = \sum_{z \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log |\mathcal{N}(z)|} \quad (2.11)$$

- **Preferential Attachment**, aspects of "rich get richer" in networks, e.g., in link prediction, to estimate the likelihood of new edges forming between high-degree nodes:

$$PA[i, j] = \deg(i) \cdot \deg(j) \quad (2.12)$$

- Edge weight itself viewable a statistic (e.g., distance/similarity between node reps.).

Hyper Trees

k -trees

Generalizations of a tree as defined as follows:

Definition 2.7.1 (k -tree)

A complete graph with $k + 1$ nodes is a k -tree. To construct a k tree with $n + 1$ nodes starting from a k -tree with n nodes, choose some size k complete sub-graph of the n -node k -tree and connect the $n + 1$ 'st node to all nodes in the k -node complete sub-graph.

k -trees

Generalizations of a tree as defined as follows:

Definition 2.7.1 (k -tree)

A complete graph with $k + 1$ nodes is a k -tree. To construct a k tree with $n + 1$ nodes starting from a k -tree with n nodes, choose some size k complete sub-graph of the n -node k -tree and connect the $n + 1$ 'st node to all nodes in the k -node complete sub-graph.

- Any complete n -graph is an $n - 1$ -tree

k -trees

Generalizations of a tree as defined as follows:

Definition 2.7.1 (k -tree)

A complete graph with $k + 1$ nodes is a k -tree. To construct a k tree with $n + 1$ nodes starting from a k -tree with n nodes, choose some size k complete sub-graph of the n -node k -tree and connect the $n + 1$ 'st node to all nodes in the k -node complete sub-graph.

- Any complete n -graph is an $n - 1$ -tree
- a regular tree is a 1-tree.

k -trees

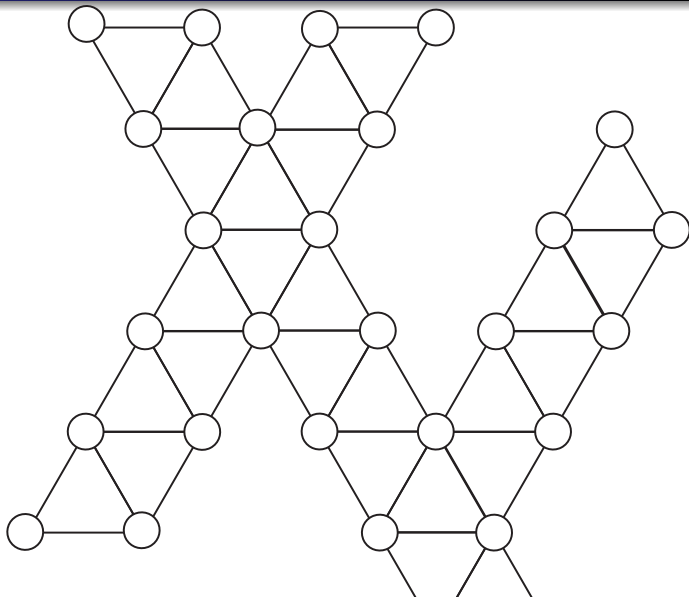
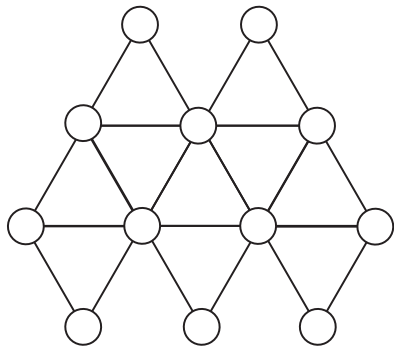
Generalizations of a tree as defined as follows:

Definition 2.7.1 (k -tree)

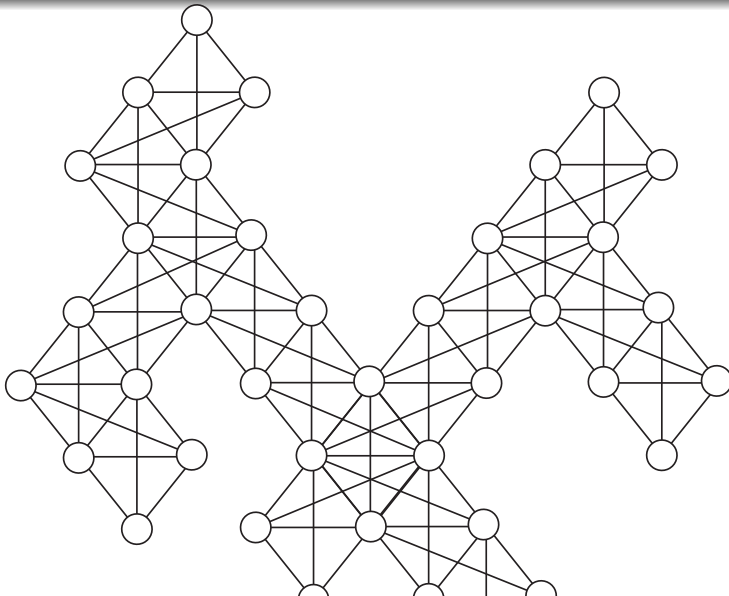
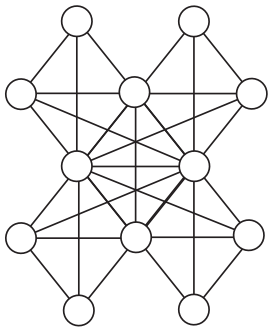
A complete graph with $k + 1$ nodes is a k -tree. To construct a k tree with $n + 1$ nodes starting from a k -tree with n nodes, choose some size k complete sub-graph of the n -node k -tree and connect the $n + 1$ 'st node to all nodes in the k -node complete sub-graph.

- Any complete n -graph is an $n - 1$ -tree
- a regular tree is a 1-tree.
- all k -trees are triangulated

Example of 2-trees



Example of 3-trees



k -trees

- In a tree, all minimal separators are size 1

k -trees

- In a tree, all minimal separators are size 1
- In a k -tree, all minimal separators are size k (and thus a k -clique).

k -trees

- In a tree, all minimal separators are size 1
- In a k -tree, all minimal separators are size k (and thus a k -clique).
- In a k -tree, all maxcliques are size $k + 1$, so maximum clique size is $k + 1$.

k -trees

- In a tree, all minimal separators are size 1
- In a k -tree, all minimal separators are size k (and thus a k -clique).
- In a k -tree, all maxcliques are size $k + 1$, so maximum clique size is $k + 1$.
- In a k -tree, complexity of inference will be $O(r^{k+1})$.

k -trees

- In a tree, all minimal separators are size 1
- In a k -tree, all minimal separators are size k (and thus a k -clique).
- In a k -tree, all maxcliques are size $k + 1$, so maximum clique size is $k + 1$.
- In a k -tree, complexity of inference will be $O(r^{k+1})$.
- even stronger:

k -trees

- In a tree, all minimal separators are size 1
- In a k -tree, all minimal separators are size k (and thus a k -clique).
- In a k -tree, all maxcliques are size $k + 1$, so maximum clique size is $k + 1$.
- In a k -tree, complexity of inference will be $O(r^{k+1})$.
- even stronger:

Lemma 2.7.2

A graph $G = (V, E)$ is a k -tree iff

- *G is connected*
- *G 's maximum clique is of size $k + 1$*
- *Every minimal separator of G is a k -clique.*

Spanning Sub-graphs

Definition 2.7.3 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G , specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.

Spanning Sub-graphs

Definition 2.7.3 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G , specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.
- A spanning k -tree of G is a k tree that can be embedded into G .

Spanning Sub-graphs

Definition 2.7.3 (Spanning Sub-graph)

A graph $G' = (V, E')$ is a spanning sub-graph of another graph $G = (V, E)$ if $E' \subseteq E$. I.e., G' is a spanning subgraph of G if G' can be embedded into G , specifically G' has the same node set as G and a subset of its edges: $E' \subseteq E$.

- Hence, a spanning tree of a graph $G = (V, E)$ is tree $T = (V, E')$ with the same node set V but E' are the edges of a tree.
- A spanning k -tree of G is a k tree that can be embedded into G .
- Why the word “spanning”? It comes from linear algebra and matroid theory, where a set is said to span a space if it reaches or covers all elements – a spanning tree similarly connects all nodes of the graph.